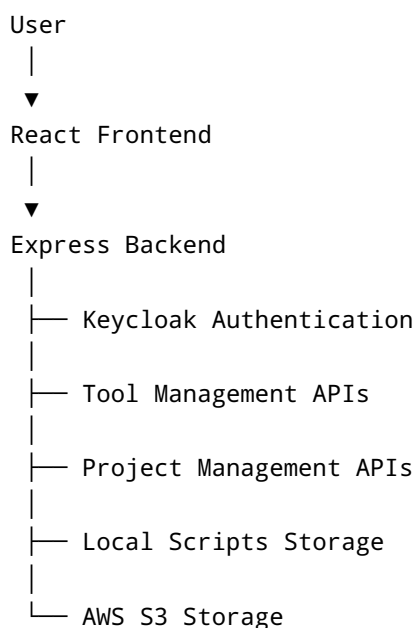


ScriptExec Portal - Technical Documentation

1. Introduction

ScriptExec Portal is a centralized web application used to manage, store, download, and execute automation tools (.exe files). The application provides secure access control using Keycloak and stores executable backups in AWS S3.

2. System Architecture



3. Technology Stack

Frontend

- React.js
- Vite
- React Router
- React Hot Toast
- Lucide React

Backend

- Node.js
- Express.js

- Multer
- JOSE
- AWS SDK

Authentication

- Keycloak

Storage

- AWS S3
- Local File System

Containerization

- Docker

4. Folder Structure

```
ScriptExec
|
├── frontend
|   ├── src
|   ├── public
|   └── package.json
|
├── backend
|   ├── scripts
|   ├── server.js
|   ├── authMiddleware.js
|   ├── tools.json
|   ├── projects.json
|   ├── history.json
|   └── package.json
|
├── docker-compose.yml
|
├── README.md
|
└── DOCUMENTATION.md
```

5. Authentication Flow

Authentication is handled through Keycloak.

Login Process

1. User opens application.
 2. Frontend redirects to Keycloak.
 3. User enters credentials.
 4. Keycloak generates JWT token.
 5. Frontend stores token.
 6. Backend validates token before processing requests.
-

Keycloak Configuration

Realm

rpa-launcher

Client

frontend-client

Roles

admin
developer
user

Creating Users

1. Login to Keycloak Admin Console.
2. Select Realm.
3. Navigate to:

Users
→ Create User

1. Enter user information.
 2. Save.
-

Setting Password

Navigate to:

Users
→ Select User
→ Credentials

Set:

Password
Temporary = OFF

Assigning Roles

Navigate to:

Users
→ Role Mapping
→ Assign Role

Assign:

admin
developer
user

6. Role Permissions

Admin

Can:

- Upload Tools
- Edit Tools
- Delete Tools
- Download Tools
- Execute Tools
- Manage Projects

Developer

Can:

- Upload Tools

- Edit Tools
- Delete Tools
- Download Tools
- Execute Tools

User

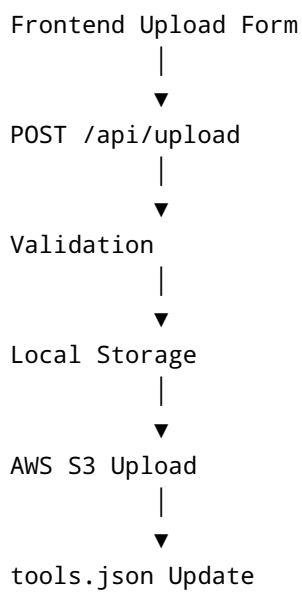
Can:

- View Tools
- Download Tools
- Execute Tools

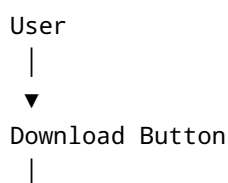
Cannot:

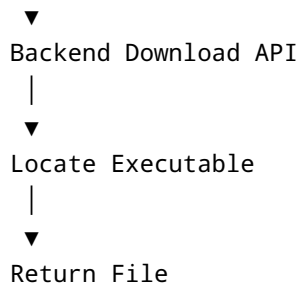
- Upload Tools
- Delete Tools

7. Tool Upload Workflow



8. Tool Download Workflow



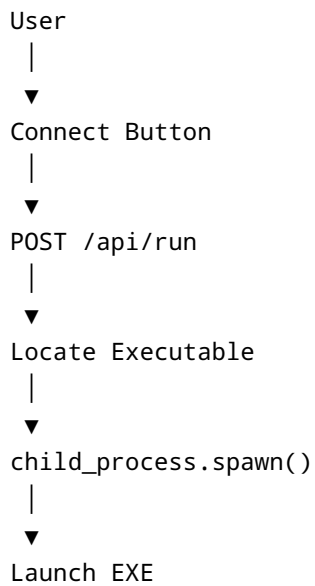


If no executable is linked:

No executable file linked to this tool

toast message is displayed.

9. Tool Execution Workflow



The executable runs as a separate Windows process.

10. AWS S3 Integration

Bucket

script-exec-tools-654

Region

ap-south-2

Purpose

- Backup uploaded executables
 - Centralized storage
 - Disaster recovery
-

11. Important Backend Files

server.js

Main backend application.

Responsible for:

- Upload APIs
 - Download APIs
 - Execution APIs
 - CRUD APIs
 - S3 Integration
-

authMiddleware.js

Responsible for:

- JWT Validation
 - Keycloak Verification
 - Route Protection
-

tools.json

Stores tool metadata.

Example:

```
{
  "name": "Tool Name",
  "project": "Project Name",
  "ticketId": "REQ-ID-1001",
```

```
"filename": "tool.exe"
}
```

projects.json

Stores project information.

history.json

Stores execution history.

12. Important Frontend Files

tools.jsx

Contains:

- Tool Cards
 - Upload Modal
 - Download Logic
 - Execution Logic
 - Delete Logic
-

keycloak.js

Contains:

- Keycloak Initialization
 - Login Configuration
-

Sidebar.jsx

Contains:

- Project Navigation
-

App.jsx

Contains:

- Application Routing
-

13. Performance Metrics

The following metrics should be measured during testing.

Metric	Description
Login Time	User login via Keycloak
Tool Load Time	Time to fetch tool list
Download Time	Time to download executable
Connect Response Time	Time after clicking Connect
EXE Launch Time	Time to open executable
Popup Initialization Time	Time until automation UI appears

14. Deployment Requirements

Backend

```
npm install
node server.js
```

Frontend

```
npm install
npm run build
```

AWS

Required:

- Access Key

- Secret Key
 - Bucket Access
-

Keycloak

Required:

- Realm
 - Client
 - Roles
 - Redirect URLs
-

15. Known Issues

spawn EFTYPE

Cause:

- Corrupted executable
- Invalid executable
- Unsupported executable format

Resolution:

- Re-upload executable
 - Verify executable runs manually
-

AccessDenied

Cause:

- Missing IAM Permissions

Resolution:

- Verify S3 Policy
-

Invalid Token

Cause:

- Expired JWT
- Incorrect Keycloak Configuration

Resolution:

- Re-login
 - Verify Client Configuration
-

16. Future Enhancements

- PostgreSQL Integration
 - Tool Versioning
 - Audit Logging
 - Analytics Dashboard
 - Centralized Monitoring
 - Remote Tool Execution
-

17. Maintenance Notes

For future development, the primary files that will require modification are:

```
backend/server.js  
backend/authMiddleware.js  
frontend/src/pages/tools.jsx  
frontend/src/keycloak.js
```

These files contain the majority of the business logic, authentication logic, and user interaction workflows.